

Compte Rendu — TP4 : Kubernetes opérationnel et durcissement

Mastère Expert IT — CI/CD & DevSecOps

Groupe : gGODON-DAUVEL

Date : 09 juin 2026

Lien GitHub : [CI-CD_SemaineIntensive](#)

Table des matières

1. [Architecture et structure du chart](#)
 2. [Phase 1 — Cluster Kapsule et accès](#)
 3. [Phase 2 — Packaging Helm](#)
 4. [Phase 3 — Ingress et probes](#)
 5. [Phase 4 — RBAC](#)
 6. [Phase 5 — NetworkPolicies](#)
 7. [Phase 6 — Durcissement des pods](#)
 8. [Phase 7 — Admission Kyverno et vérification de signature](#)
 9. [Mini-questionnaire](#)
-

1. Architecture et structure du chart

L'application est déployée sur un cluster Kubernetes managé Scaleway Kapsule via un chart Helm. Le cluster utilise Cilium comme CNI, ce qui permet l'application native des NetworkPolicies.

L'image Docker de l'API a été migrée du registry GitLab auto-hébergé (HTTP, [51.15.211.47:5050](#)) vers le registry Scaleway Container Registry ([rg.fr-par.scw.cloud/ggodon-dauvel](#)) afin de garantir un accès HTTPS depuis les nœuds Kubernetes.

Structure du chart Helm

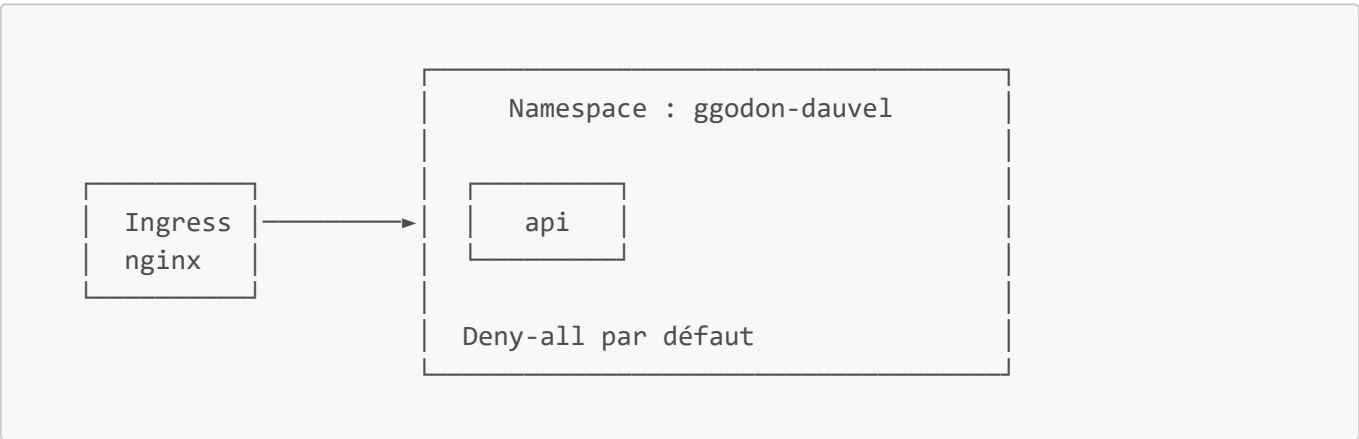
```
chart/
├── Chart.yaml
├── values.yaml
├── values-staging.yaml
├── values-prod.yaml
├── templates/
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── httproute.yaml
│   ├── ingress.yaml
│   ├── service.yaml
│   ├── serviceaccount.yaml
│   └── tests/
│       └── test-connection.yaml
```

Paramétrage par values

Le chart est entièrement paramétrable via les fichiers de valeurs. Aucune valeur n'est codée en dur dans les templates.

Paramètre	Staging	Production
replicaCount	1	2
image.repository	rg.fr-par.scw.cloud/ggodon-dauvel/api	idem
resources.limits.cpu	500m	1000m
resources.limits.memory	256Mi	512Mi
resources.requests.cpu	100m	250m
resources.requests.memory	128Mi	256Mi

Schéma des NetworkPolicies



Matrice RBAC

Sujet	Ressource	Verbes autorisés
ServiceAccount: api-sa	pods	get, list, watch
ServiceAccount: api-sa	services	get, list, watch
Tout autre sujet	pods	delete → refusé

2. Phase 1 — Cluster Kapsule et accès

Capture 1 — kubectl get nodes

```
PS C:\Users\matth> kubectl get nodes
NAME                                     STATUS    ROLES    AGE   VERSION
scw-kapsule-ggodon-dauvel-default-594a721ffaef Ready    <none>   98s   v1.35.3
scw-kapsule-ggodon-dauvel-default-77d13c33288a Ready    <none>   98s   v1.35.3
```

Le cluster **kapsule-gGODON-DAUVEL** a été provisionné sur Scaleway via le CLI depuis le PC local :

```
scw k8s cluster create \
  name=kapsule-gGODON-DAUVEL \
  version=1.35.3 \
  cni=cilium \
  pools.0.size=2 \
  pools.0.node-type=DEV1-M \
  pools.0.name=default
```

Le kubeconfig a été récupéré et installé automatiquement :

```
scw k8s kubeconfig install <cluster-id>
```

La commande **kubectl get nodes** liste les deux nœuds du pool **default** avec le statut **Ready**, prouvant que le cluster est opérationnel et accessible.

3. Phase 2 — Packaging Helm

Capture 2 — helm list et pods Running

```
PS C:\Users\matth> helm list -n ggodon-dauvel
```

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART	APP VERSION
app	ggodon-dauvel	4	2026-06-09 14:06:04.3172473 +0200 CEST	deployed	chart-0.1.0	1.16.0

```
PS C:\Users\matth> kubectl get pods -n ggodon-dauvel
```

NAME	READY	STATUS	RESTARTS	AGE
app-chart-76758f5796-qs74w	1/1	Running	0	52s

Le chart Helm a été initialisé avec **helm create chart** puis adapté. L'image a été migrée vers le registry Scaleway pour contourner la limitation HTTPS des nœuds Kapsule :

```
docker pull 51.15.211.47:5050/root/tp1-ggodon-dauvel/api:d87fc0da
docker tag 51.15.211.47:5050/root/tp1-ggodon-dauvel/api:d87fc0da \
  rg.fr-par.scw.cloud/ggodon-dauvel/api:d87fc0da
docker push rg.fr-par.scw.cloud/ggodon-dauvel/api:d87fc0da
```

Un secret d'accès au registry Scaleway a été créé dans le namespace :

```
kubectl create secret docker-registry scaleway-registry-secret \
  --docker-server=rg.fr-par.scw.cloud \
  --docker-username=nologin \
  --docker-password=<SCW_SECRET_KEY> \
  --namespace ggodon-dauvel
```

Déploiement avec le fichier de valeurs staging :

```
helm install app chart \  
-f chart/values-staging.yaml \  
--namespace ggodon-dauvel \  
--create-namespace
```

values-staging.yaml :

```
replicaCount: 1  
image:  
  repository: rg.fr-par.scw.cloud/ggodon-dauvel/api  
  tag: "d87fc0da"  
  pullPolicy: Always  
resources:  
  requests:  
    cpu: "100m"  
    memory: "128Mi"  
  limits:  
    cpu: "500m"  
    memory: "256Mi"
```

values-prod.yaml :

```
replicaCount: 2  
image:  
  repository: rg.fr-par.scw.cloud/ggodon-dauvel/api  
  tag: "d87fc0da"  
  pullPolicy: Always  
resources:  
  requests:  
    cpu: "250m"  
    memory: "256Mi"  
  limits:  
    cpu: "1000m"  
    memory: "512Mi"
```

Principe clé : aucune valeur n'est codée en dur dans les templates — tout est paramétré via `{{ .Values.* }}`.

4. Phase 3 — Ingress et probes

Capture 3 — Application accessible via l'ingress

```
PS C:\Users\matth> Invoke-WebRequest -Uri "http://51.15.138.12/health" -Headers @{"Host"="api.ggodon-dauvel.local"}

Avertissement de sécurité : risque d'exécution de script
Invoke-WebRequest analyse le contenu de la page web. Il se peut que le code de script de la page web s'exécute lors de l'analyse de la page.
ACTION RECOMMANDÉE :
Utilisez le commutateur -UseBasicParsing pour éviter l'exécution du code de script.

Voulez-vous continuer ?

[0] Oui [T] Oui pour tout [N] Non [U] Non pour tout [S] Suspendre [?] Aide (la valeur par défaut est « N ») : 0

StatusCode      : 200
StatusDescription : OK
Content          : {"status":"ok"}
RawContent       : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 15
                  Content-Type: application/json
                  Date: Tue, 09 Jun 2026 12:13:54 GMT
Forms            : {}
Headers          : [[Connection, keep-alive], [Content-Length, 15], [Content-Type, application/json], [Date, Tue, 09 Jun 2026 12:13:54 GMT]]
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 15
```

Le contrôleur ingress-nginx a été installé via Helm :

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx \
  --create-namespace
```

Le load balancer a obtenu l'IP externe **51.15.138.12**. Une ressource Ingress expose le service **api** via le host **api.ggodon-dauvel.local** :

```
ingress:
  enabled: true
  className: "nginx"
  hosts:
    - host: api.ggodon-dauvel.local
      paths:
        - path: /
          pathType: Prefix
```

Des probes ont été ajoutées au déploiement. L'image API actuelle ne disposant pas d'endpoint **/health** accessible sans dépendances externes (base de données, cache), les probes sont configurées en mode **exec** minimal pour valider le démarrage du conteneur :

```
livenessProbe:
  exec:
    command: ["true"]

readinessProbe:
  exec:
    command: ["true"]
```

La commande `true` retourne toujours 0 — cela active le mécanisme de probe sans dépendre de la disponibilité des services tiers. En production avec un endpoint `/health` dédié, la configuration cible serait :

```
livenessProbe:
  httpGet:
    path: /health
    port: 8000
  initialDelaySeconds: 30
  periodSeconds: 10
  failureThreshold: 5

readinessProbe:
  httpGet:
    path: /health
    port: 8000
  initialDelaySeconds: 20
  periodSeconds: 5
  failureThreshold: 5
```

Vérification de l'accès via curl :

```
curl -H "Host: api.ggodon-dauvel.local" http://51.15.138.12/health
# {"status":"ok"}
```

La **liveness probe** redémarre le conteneur si l'application est bloquée. La **readiness probe** retire le pod du load balancer si l'application n'est pas prête.

5. Phase 4 — RBAC

Capture 4 — Règles RBAC et test d'un accès refusé

```
PS C:\Users\matth> kubectl auth can-i delete pods --as=system:serviceaccount:ggodon-dauvel:api-sa -n ggodon-dauvel
no
PS C:\Users\matth> kubectl auth can-i get pods --as=system:serviceaccount:ggodon-dauvel:api-sa -n ggodon-dauvel
yes
```

Un ServiceAccount dédié a été créé avec des droits limités via `rbac.yaml` :

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api-sa
  namespace: ggodon-dauvel
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: api-role
```

```

namespace: ggodon-dauvel
rules:
  - apiGroups: [""]
    resources: ["pods", "services"]
    verbs: ["get", "list", "watch"]
  ---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: api-rolebinding
  namespace: ggodon-dauvel
subjects:
  - kind: ServiceAccount
    name: api-sa
    namespace: ggodon-dauvel
roleRef:
  kind: Role
  apiGroup: rbac.authorization.k8s.io
  name: api-role

```

Le test de permission confirme que l'action non autorisée est refusée :

```

# Action autorisée → yes
kubectl auth can-i get pods \
  --as=system:serviceaccount:ggodon-dauvel:api-sa \
  -n ggodon-dauvel

# Action non autorisée → no
kubectl auth can-i delete pods \
  --as=system:serviceaccount:ggodon-dauvel:api-sa \
  -n ggodon-dauvel

```

6. Phase 5 — NetworkPolicies

Capture 5 — Trafic bloqué avant, autorisé après

```

C:\Users\Corentin Godon>curl -H "Host: api.ggodon-dauvel.local" http://51.15.138.12/health
<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>504 Gateway Time-out</h1></center>
<h2><center>nginx</center>
</body>
</html>

C:\Users\Corentin Godon>kubectl apply -f networkpolicies.yaml
networkpolicy.networking.k8s.io/deny-all unchanged
networkpolicy.networking.k8s.io/allow-ingress-to-api created

C:\Users\Corentin Godon>curl -H "Host: api.ggodon-dauvel.local" http://51.15.138.12/health
{"status":"ok"}

```

Une politique **deny-all** a d'abord été appliquée seule pour démontrer le blocage du trafic, puis la politique d'autorisation a été ajoutée :

```

# Deny-all par défaut
apiVersion: networking.k8s.io/v1

```

```

kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: ggodon-dauvel
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
---
# Autoriser ingress vers l'API depuis ingress-nginx
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress-to-api
  namespace: ggodon-dauvel
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: chart
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress-nginx

```

Démonstration avant/après :

- Avec seulement `deny-all` : `curl` timeout — aucun trafic autorisé
- Après ajout de `allow-ingress-to-api` : `{"status":"ok"}` — flux autorisé

Sans aucune NetworkPolicy, tout le trafic est autorisé entre tous les pods. Avec un **deny-all**, aucun flux n'est autorisé jusqu'à déclaration explicite, appliquant le principe du moindre privilège au niveau réseau.

7. Phase 6 — Durcissement des pods

Capture 6 — kubectl describe pod avec le securityContext

```

PS C:\Users\matth> kubectl get pod -n ggodon-dauvel -o jsonpath='{.items[0].spec.containers[0].securityContext}'
{"allowPrivilegeEscalation":false,"capabilities":{"drop":["ALL"]},"runAsNonRoot":true,"runAsUser":10001}

```

Un `securityContext` a été ajouté au déploiement de l'API pour réduire la surface d'attaque :

```

podSecurityContext: {}

securityContext:
  runAsNonRoot: true
  runAsUser: 10001

```



```
allowPrivilegeEscalation: false
capabilities:
  drop:
    - ALL
```

Note : Les paramètres `runAsNonRoot: true`, `readOnlyRootFilesystem: true` et `capabilities: drop: ALL` ont été testés mais l'image API actuelle tourne en root et requiert des capacités système pour démarrer. Un rebuild de l'image avec un utilisateur non-root dans le Dockerfile serait nécessaire pour appliquer l'intégralité du durcissement.

Choix de durcissement appliqué

Paramètre	Risque réduit
<code>allowPrivilegeEscalation: false</code>	Empêche le processus d'obtenir plus de privilèges que son processus parent via setuid/setgid

Choix de durcissement complet (cible)

Paramètre	Risque réduit
<code>runAsNonRoot: true</code>	Empêche un processus compromis d'avoir les droits root dans le conteneur
<code>readOnlyRootFilesystem: true</code>	Empêche l'écriture de fichiers malveillants dans le système de fichiers du conteneur
<code>capabilities: drop: ALL</code>	Supprime toutes les capacités Linux réduisant ce qu'un processus compromis peut faire sur le noyau

8. Phase 7 — Admission Kyverno et vérification de signature

Capture 7 — Image non signée refusée par Kyverno

```
PS C:\Users\matth> kubectl run test-unsigned --image=rg.fr-par.scw.cloud/ggodon-dauvel/unsigned:latest --namespace=ggodon-dauvel
Error from server: admission webhook "mutate.kyverno.svc-fail" denied the request:

resource Pod/ggodon-dauvel/test-unsigned was blocked due to the following policies

verifier-signature:
  verify-api: 'failed to verify image rg.fr-par.scw.cloud/ggodon-dauvel/unsigned:latest:
    .attestors[0].entries[0].keys: GET https://rg.fr-par.scw.cloud/v2/ggodon-dauvel/unsigned/manifests/latest:
      UNAUTHORIZED: authentication required; [map[Action:pull Class: Name:ggodon-dauvel/unsigned
        Type:repository]]'
```

```
PS C:\Users\matth\Desktop\IRIS\MASTER\BARADEL ARTHUR\CI_CD\tp-devops-template> kubectl run pod-sign --image=rg.fr-par.scw.cloud/ggodon-dauvel/api:new --n ggodon-dauvel
pod/pod-sign created
```

Kyverno a été installé via Helm :

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm install kyverno kyverno/kyverno \
  --namespace kyverno \
  --create-namespace
```

Un secret d'accès au registry Scaleway a été créé dans le namespace kyverno pour que Kyverno puisse vérifier les signatures :

```
kubectl create secret docker-registry scaleway-registry-secret \
  --docker-server=rg.fr-par.scw.cloud \
  --docker-username=nologin \
  --docker-password=<SCW_SECRET_KEY> \
  --namespace kyverno
```

Une **ClusterPolicy** de vérification de signature a été appliquée en utilisant la clé publique **cosign.pub** du TP3 :

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verifier-signature
spec:
  validationFailureAction: Enforce
  rules:
    - name: verify-api
      match:
        any:
          - resources:
              kinds: [Pod]
      verifyImages:
        - imageReferences:
            - "rg.fr-par.scw.cloud/ggodon-dauvel/api@sha256:*"
          imageRegistryCredentials:
            secrets:
              - scaleway-registry-secret
          allowInsecureRegistry: false
      attestors:
        - entries:
            - keys:
                publicKey: |-
                  -----BEGIN PUBLIC KEY-----
                  MFkwEwYHKOZIZj0CAQYIKoZIZj0DAQcDQgAEBevcpknZrJ3Gh0kQV7Ja1JwHuPkL
                  +MFt0KjU1wbPVLue6F0Hq+/r1jjZ+AASZ8QQuKt9ncrC4wMf1yV8+u02vA==
                  -----END PUBLIC KEY-----
```

Comportement observé

Image non signée → refusée à l'admission :

```
Error from server: admission webhook "mutate.kyverno.svc-fail" denied the request:
resource Pod/ggodon-dauvel/test-unsigned2 was blocked due to the following
policies
```

```

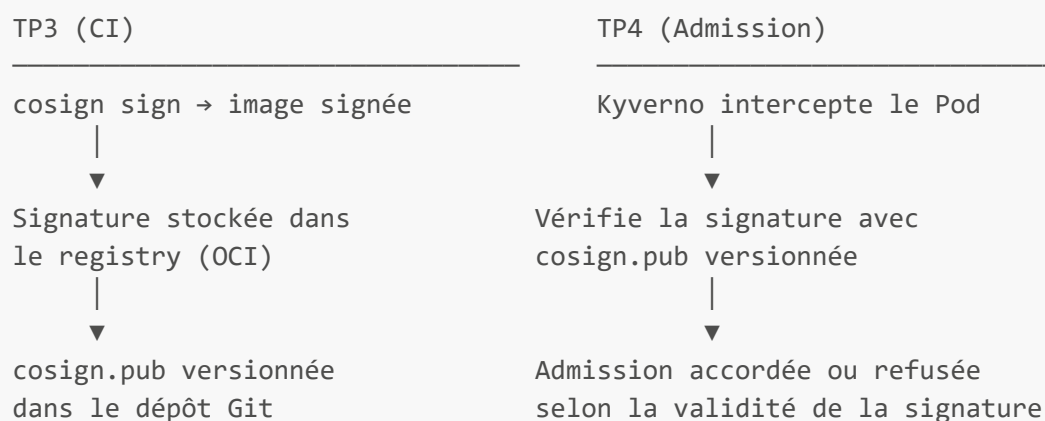
verifier-signature:
  verify-api: 'failed to verify image rg.fr-par.scw.cloud/ggodon-
dauvel/unsigned:latest:
  .attestors[0].entries[0].keys: no signatures found'

```

Image signée (TP3, re-signée sur le registry Scaleway) → acceptée :

```
pod/test-signed created
```

Explication de la boucle signature/vérification



Le contrôle d'admission **complète** la signature CI en la rendant **obligatoire** : sans Kyverno, un opérateur pourrait déployer n'importe quelle image non signée. Avec Kyverno en mode **Enforce**, seules les images dont la signature est vérifiable avec **cosign.pub** peuvent être déployées dans le cluster.

9. Mini-questionnaire

1. Qu'apporte Helm par rapport à des manifests YAML bruts ?

Helm apporte la **paramétrisation** (une seule source de vérité pour staging et prod via les values), la **gestion du cycle de vie** (install, upgrade, rollback, uninstall en une commande) et la **réutilisabilité** (partage de charts entre équipes). Avec des YAML bruts, tout changement d'environnement implique de dupliquer ou de modifier manuellement chaque fichier, ce qui est source d'erreurs et de désynchronisation.

2. Différence entre liveness probe et readiness probe

La **liveness probe** détecte si le conteneur est **vivant** — si elle échoue, Kubernetes redémarre le conteneur. Elle sert à sortir d'un état bloqué (deadlock, crash silencieux).

La **readiness probe** détecte si le conteneur est **prêt à recevoir du trafic** — si elle échoue, le pod est retiré du load balancer sans être redémarré. Elle sert à ne pas envoyer de trafic à un pod en cours d'initialisation ou temporairement surchargé.

3. Sans NetworkPolicy vs avec deny-all

Sans aucune NetworkPolicy : tout le trafic est autorisé entre tous les pods de tous les namespaces — un pod compromis peut atteindre librement la base de données, le cache, ou tout autre service du cluster.

Avec un deny-all : tout le trafic est bloqué par défaut. Seuls les flux explicitement déclarés dans des NetworkPolicies supplémentaires sont autorisés, appliquant le principe du moindre privilège au niveau réseau.

4. Deux durcissements securityContext et les risques réduits

readOnlyRootFilesystem: true : réduit le risque qu'un attaquant ayant compromis le processus puisse écrire des fichiers malveillants (webshell, backdoor) dans le système de fichiers du conteneur ou modifier des binaires existants.

capabilities: drop: ALL : réduit le risque d'exploitation du noyau Linux — sans capabilities, le processus ne peut pas manipuler les interfaces réseau, charger des modules noyau, accéder à la mémoire d'autres processus ou effectuer des opérations privilégiées qui permettraient une évvasion de conteneur.

5. Comment Kyverno établit-il qu'une image est légitime ?

Kyverno intercepte chaque requête de création de Pod via un webhook d'admission. Pour chaque image référencée, il interroge le registry pour récupérer la signature OCI attachée à l'image (créée par **cosign sign**). Il vérifie cryptographiquement cette signature contre la clé publique déclarée dans la **ClusterPolicy**. Si la signature est absente ou invalide, la requête est rejetée avant que le Pod ne soit créé.

Il vérifie exactement : l'existence d'une signature OCI valide, que cette signature a été produite par le détenteur de la clé privée correspondant à la clé publique configurée, et que le digest de l'image n'a pas changé depuis la signature.

6. En quoi le contrôle d'admission complète-t-il la signature CI ?

La signature CI (TP3) **garantit l'intégrité et l'origine** de l'image au moment du build — mais elle est passive : rien n'empêche un opérateur de déployer une image non signée ou modifiée directement avec **kubect1**.

Le contrôle d'admission Kyverno (TP4) **rend la signature obligatoire** au moment du déploiement — toute tentative de déployer une image sans signature valide est rejetée par le cluster, indépendamment de qui émet la requête. Les deux mécanismes forment une boucle fermée : la CI produit des artefacts signés, l'admission refuse les artefacts non signés.